

LUIZ FERNANDO CUNHA MAIA

Curso de Engenharia de Software
Disciplina: Redes de computadores

PROGRAMAÇÃO COM SOCKETS EM PYTHON
UDP, TCP iterativo e TCP concorrente

1 INTRODUÇÃO

Este relatório apresenta seis programas em Python: clientes e servidores UDP, TCP iterativo e TCP concorrente. São reproduzidos os códigos do repositório e as capturas de execução fornecidas, acompanhados de explicações do funcionamento.

2 COMUNICAÇÃO UDP

2.1 Cliente: UDPClient.py

```
from socket import *

# Endereço alterado para localhost/127.0.0.1 conforme orientação
serverName = '127.0.0.1'
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_DGRAM)

message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))

modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())

clientSocket.close()
```

O cliente cria um socket IPv4 de datagramas (SOCK_DGRAM), lê o texto e o envia para 127.0.0.1:12000 com sendto(). Em seguida, recvfrom(2048) aguarda a resposta, que é decodificada e exibida. Por fim, o socket é fechado.

2.2 Servidor: UDPServer.py

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))

print('The server is ready to receive')

while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

O servidor associa o socket à porta 12000 em todas as interfaces locais. Em um laço contínuo, recebe o datagrama e o endereço do remetente, converte o texto para maiúsculas com upper() e devolve o resultado com sendto().

2.3 Execução do UDP

Primeiro, executa-se `python UDP/UDPServer.py`. Em outro terminal, executa-se `python UDP/UDPClient.py`. Os comandos são executados a partir da pasta principal do projeto.

Figura 1 - Servidor UDP aguardando datagramas

```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/UDP/UDPServer.py
The server is ready to receive
█
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

Figura 2 - Cliente UDP: envio e resposta

```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/UDP/UDPClient.py
Input lowercase sentence: nhe nhe
NHE NHE
PS C:\Users\ferna\REDES-SOCKETS> █
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

Na captura, o cliente envia “nhe nhe” e recebe “NHE NHE”. A mensagem “The server is ready to receive” indica que o servidor iniciou e está aguardando dados. O servidor permanece ativo após responder.

O UDP envia datagramas sem estabelecer uma conexão prévia e não garante entrega nem ordem. Neste exemplo, o cliente fica aguardando a resposta porque não foi configurado um tempo limite de espera.

3 COMUNICAÇÃO TCP ITERATIVA

3.1 Cliente: TCPClient.py

```
from socket import *

# Endereço alterado para localhost/127.0.0.1 conforme orientação
serverName = '127.0.0.1'
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))

sentence = input('Input lowercase sentence: ')
clientSocket.send(sentence.encode())

modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())

clientSocket.close()
```

O cliente cria um socket IPv4 de fluxo (SOCK_STREAM) e estabelece a conexão com 127.0.0.1:12000 por connect(). Depois, envia o texto, recebe até 1024 bytes da resposta, exibe o resultado e fecha a conexão.

3.2 Servidor: TCPServer.py

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)

print('The server is ready to receive')

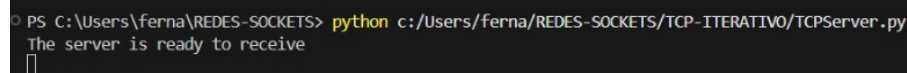
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

O servidor executa bind() e listen() para aceitar conexões. A cada accept(), obtém um socket específico para o cliente, recebe o texto, converte-o para maiúsculas e envia a resposta. Esse socket é fechado antes do atendimento seguinte, caracterizando o funcionamento iterativo.

3.3 Execução do TCP iterativo

Inicia-se python TCP-ITERATIVO/TCPServer.py e, em outro terminal, python TCP-ITERATIVO/TCPClient.py.

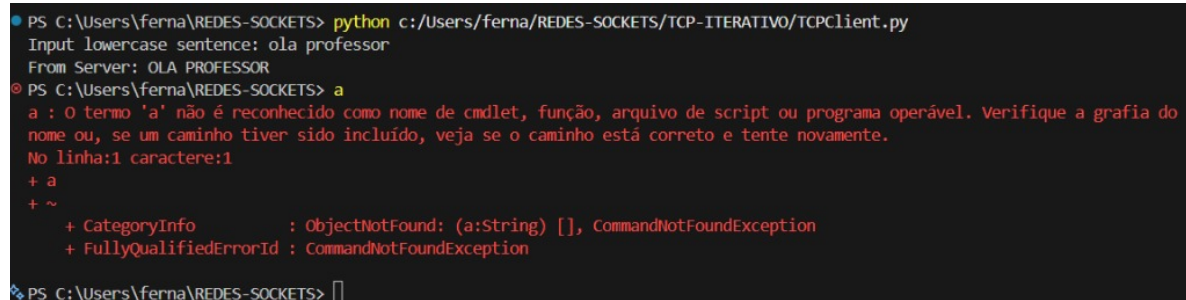
Figura 3 - Servidor TCP iterativo aguardando conexão



```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/TCP-ITERATIVO/TCPServer.py
The server is ready to receive
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

Figura 4 - Cliente TCP iterativo: envio e resposta



```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/TCP-ITERATIVO/TCPClient.py
Input lowercase sentence: ola professor
From Server: OLA PROFESSOR
PS C:\Users\ferna\REDES-SOCKETS> a
a : O termo 'a' não é reconhecido como nome de cmdlet, função, arquivo de script ou programa operável. Verifique a grafia do
nome ou, se um caminho tiver sido incluído, veja se o caminho está correto e tente novamente.
No linha:1 caractere:1
+ a
+ ~
+ CategoryInfo          : ObjectNotFound: (a:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
PS C:\Users\ferna\REDES-SOCKETS>
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

O cliente envia “ola professor” e recebe “From Server: OLA PROFESSOR”, confirmando a conversão. O erro em vermelho ocorre depois do encerramento do cliente: a letra “a” foi digitada como comando no PowerShell e não foi reconhecida pelo terminal.

O atendimento é sequencial: enquanto recebe e processa os dados de uma conexão, o servidor não chama accept() para a próxima. O argumento de listen(1) se refere à fila de conexões pendentes, não à quantidade total de clientes que podem usar o programa.

4 COMUNICAÇÃO TCP CONCORRENTE

4.1 Servidor: TCPServerConc.py

```
# -*- coding: utf-8 -*-
import socket
import _thread

HOST = '127.0.0.1' # Endereço IP do Servidor
PORT = 50000      # Porta em que o Servidor está escutando

def conectado(con, cliente):
    print('\nCliente conectado:', cliente)
    while True:
        msg = con.recv(1024)
        if not msg:
            break
        print('\nCliente...:', cliente)
        print('Mensagem.:', msg.decode())

        print('\nFinalizando conexao do cliente', cliente)
        con.close()
        _thread.exit()

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)

tcp.bind(orig)
tcp.listen(1)

print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta', PORT)

while True:
    con, cliente = tcp.accept()
    print('\nNova thread iniciada para essa conexão')
    _thread.start_new_thread(conectado, (con, cliente))

tcp.close()
```

O servidor escuta em 127.0.0.1:50000. Para cada conexão aceita, `_thread.start_new_thread()` inicia uma thread que executa `conectado()`. Assim, o laço principal pode aceitar novos clientes enquanto as threads recebem dados das conexões existentes.

Cada thread exibe o endereço do cliente e a mensagem recebida. Quando `recv()` retorna dados vazios, a thread encerra o laço, fecha o socket e termina. O servidor apenas registra as mensagens; não envia respostas nem converte o texto para maiúsculas.

4.2 Cliente: TCPClientConc.py

```
import socket

HOST = '127.0.0.1' # Endereço IP do Servidor
PORT = 50000      # Porta em que o Servidor está

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)
tcp.connect(destino)

print('\nDigite suas mensagens')
print('Para sair use CTRL+X\n')

mensagem = input()

# Permanece enviando mensagens até o usuário pressionar CTRL+X (código ASCII \x18)
while mensagem != '\x18':
    tcp.send(mensagem.encode())
    mensagem = input()

tcp.close()
```

O cliente estabelece uma conexão TCP com 127.0.0.1:50000. Em seguida, lê e envia mensagens sucessivas pela mesma conexão. Não existe `recv()` neste programa: o resultado é observado no terminal do servidor.

O laço termina quando `input()` recebe o caractere de controle de código hexadecimal 18 (CTRL+X), conforme a instrução exibida pelo programa. Ao sair do laço, `tcp.close()` encerra a conexão. A entrada desse caractere depende do comportamento do terminal.

4.3 Procedimento de execução

Executa-se `python TCP-CONCORRENTE/TCPServerConc.py` e, em outro terminal, `python TCP-CONCORRENTE/TCPClientConc.py`. Para exercitar o atendimento concorrente, podem ser abertos outros clientes em terminais separados.

As capturas fornecidas mostram uma conexão e o envio da mensagem “Oi”. A criação de uma thread por conexão está demonstrada no código; as imagens não documentam um teste com vários clientes simultâneos.

4.4 Capturas do TCP concorrente

Figura 5 - Cliente TCP concorrente enviando a mensagem “Oi”

```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/TCP-CONCORRENTE/TCPClientConc.py

Digite suas mensagens
Para sair use CTRL+X

Oi
█
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

Figura 6 - Servidor TCP concorrente recebendo a mensagem

```
PS C:\Users\ferna\REDES-SOCKETS> python c:/Users/ferna/REDES-SOCKETS/TCP-CONCORRENTE/TCPServerConc.py

Servidor TCP concorrente iniciado no IP 127.0.0.1 na porta 50000

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 54469)

Cliente.: ('127.0.0.1', 54469)
Mensagem.: Oi
█
```

Fonte: captura fornecida no repositório (recorte da área do terminal).

O servidor informa a criação da thread e identifica o cliente pelo par IP e porta: 127.0.0.1 e 54469. A porta 54469 pertence ao cliente nessa execução; o servidor continua escutando na porta 50000. O registro “Mensagem.: Oi” confirma o recebimento.

O cliente permanece disponível para novas entradas após o envio. Enquanto isso, a thread do servidor aguarda mais dados dessa conexão e o laço principal permanece disponível para aceitar outras conexões.

5 CONCLUSÃO

Os programas demonstram três formas de comunicação por sockets. No UDP, a troca ocorre por datagramas. No TCP iterativo, uma conexão é atendida por vez. No TCP concorrente, uma thread atende cada conexão, permitindo que diferentes clientes mantenham sessões ao mesmo tempo.

As capturas registram a conversão para maiúsculas nos exemplos UDP e TCP iterativo e o recebimento de texto no TCP concorrente. Os códigos foram preservados; as explicações descrevem o comportamento efetivamente implementado.

Os exemplos TCP usam `send()` e uma leitura por `recv()`, sem um protocolo que delimite mensagens. Como TCP transporta um fluxo de bytes, uma leitura pode conter apenas parte de um envio ou dados de vários envios. Para aplicações mais completas, são necessários tratamento de envio parcial, delimitação de mensagens e tratamento de erros.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 14724:2024: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

PROGRAMAÇÃO com Sockets em Python: UDP, TCP e TCP Concorrente. Repositório REDES-SOCKETS. [S. l.: s. n.], [s. d.]. Material fornecido: README.md, seis arquivos Python e seis capturas JPG.

<https://github.com/Luizfmaia10-dev/REDES-SOCKETS>